UNIVERSIDAD TECNOLÓGICA DE TORREÓN

EQUIPO:
1

TÍTULO:
"ANTEPROYECTO - UTT CONNECT"

ALUMNOS:
LUNA CHAVEZ ERNESTO
SALDÍVAR MARTÍNEZ SARA ELIZABETH
MEREDITH JUDITH VALDIVIA HERNÁNDEZ
ANIBAL JAZIEL HERNÁNDEZ FRAYRE
GALVÁN PÉREZ CÉSAR ALBERTO

MAESTRO(A):
MCC. DEL TORO ROBLEDO, JESÚS

No. MATRÍCULA:
24170035
24170068
24170085
24170069
24330002

FECHA:
15 DE SEPTIEMBRE DEL 2026

INDICE

PROBLEMÁTICA **4**
OBJETIVO GENERAL **4**
OBJETIVOS ESPECÍFICOS **4**
JUSTIFICACIÓN **5**
ALCANCES Y LIMITACIONES **5**
1. Alcance del Proyecto 5
2. Limitaciones del Proyecto 6
PRODUCTO MÍNIMO VIABLE (MVP) **7**
REQUISITOS FUNCIONALES (RF) **8**
Módulo de Interfaz Pública (Landing Page): 8
Módulo de Autenticación y Seguridad: 8
Módulo de Administración (Rol: Admin): 9
Módulo del Muro de Avisos (Foro): 10
Módulo de Soporte Directo (Chat): 10
Módulo de Notificaciones: 10
REQUISITOS NO FUNCIONALES (RNF) **10**
Módulo de Interfaz y Experiencia de Usuario (UI/UX): 10
Módulo de Arquitectura y Rendimiento: 11
Módulo de Ciberseguridad y Privacidad: 11
ANÁLISIS DE RIESGOS **12**
EJEMPLO MATRIZ DE ANÁLISIS DE RIESGOS (5x5): 15
METODOLOGÍA DE DESARROLLO **15**
Marco Metodológico: Enfoque Híbrido Scrumban (Scrum + Kanban) 15
1. Implementación del Marco Scrum (Estructura Incremental y Temporal) 16
● Delimitación de Roles: 16
● Ceremonias del Ciclo de Vida: 16
2. Implementación de Kanban (Control Visual del Flujo de Trabajo) 16
3. Sinergia y Justificación de la Metodología 17
ROLES DE LOS INTEGRANTES **17**
TAREAS DEL PROYECTO **18**
Muro de Avisos y Roles (Octubre 16 - 31) 18
Chat y Notificaciones (Noviembre 1 - 15) 21
Estabilización, QA y Cierre (Noviembre 16 - Diciembre 10) 24
CRONOGRAMA **26**
ESTRUCTURA Y ARQUITECTURA DEL SISTEMA **26**
1. Arquitectura en Tres Capas 26
2. Patrón de Aplicación de Página Única (Single Page Application — SPA) 27
TECNOLOGÍAS **28**
DESCRIPCIÓN DE CONTENIDOS (WIREFRAME) **29**
Tabla 1: Inventario de Recursos Textuales (Titulares, Microcopys y Formularios) 29
Tabla 2: Inventario de Recursos Visuales e Iconografía 32

Tabla 3: Fuentes Tipográficas y Fuentes de Información 33
Tabla 4: Tono de Comunicación y Criterios de Selección 34
Tabla 5: Matriz de Licencias y Derechos de Uso 35

PROBLEMÁTICA

En la Universidad Tecnológica de Torreón (UTT), se ha detectado una ineficiencia en los canales de comunicación institucionales que vinculan a la dirección, los departamentos administrativos, el cuerpo docente y la comunidad estudiantil. Esta deficiencia en la coordinación interdepartamental e institucional genera ambigüedad recurrente respecto a los lineamientos, requisitos y procedimientos operativos para la realización de trámites académicos y administrativos (tales como inscripciones, servicio social, estadías profesionales, pagos, titulación y gestión de becas).

Como resultado, la comunidad estudiantil enfrenta un flujo de información incompleto o contradictorio, lo que deriva en desplazamientos presenciales innecesarios a las instalaciones de la universidad. Dicha problemática genera un impacto socioeconómico y operacional negativo, caracterizado por el uso ineficiente del tiempo, costos imprevistos en transporte y reprografía, así como el rechazo recurrente de solicitudes debido al incumplimiento de formatos o plazos no clarificados oportunamente. En consecuencia, la carencia de un sistema de comunicación centralizado y oficial compromete la resolución eficaz de dudas, afectando la operatividad institucional y el trayecto académico del alumnado.

OBJETIVO GENERAL

Diseñar, desarrollar e implementar una plataforma web unificada de comunicación y consulta institucional para la Universidad Tecnológica de Torreón (UTT), orientada a centralizar y categorizar la difusión de avisos oficiales en tiempo real, así como a agilizar la gestión de trámites y solicitudes mediante un tablero informativo estructurado y un sistema de mensajería directa interdepartamental, con el propósito de optimizar la interacción entre las áreas administrativas, el personal docente y la comunidad estudiantil, reduciendo los tiempos de respuesta y la ambigüedad en los procesos institucionales.

OBJETIVOS ESPECÍFICOS

1. **Arquitectura y Publicación:** Diseñar el módulo de muro informativo categorizado, restringiendo los permisos de creación y gestión de contenido exclusivamente a usuarios con rol administrativo validado.

2. **Seguridad y Control de Usuarios:** Implementar el mecanismo de autenticación del sistema vinculado al dominio de correo institucional (@uttcampus.edu.mx) para garantizar un entorno seguro y exclusivo para la comunidad universitaria.

3. **Módulo de Comunicación:** Desarrollar el canal de atención personalizada mediante un sistema de chat individual departamental que permita el seguimiento directo de trámites y consultas.

4. **Validación e Impacto:** Validar la usabilidad de la plataforma y su efectividad en la optimización de los tiempos de atención, la reducción de trámites extemporáneos y la disminución de desplazamientos innecesarios del alumnado.

JUSTIFICACIÓN

La ausencia de un canal de comunicación directo y estandarizado en la Universidad Tecnológica de Torreón (UTT) genera un impacto negativo directo en la comunidad estudiantil, manifestado en la duplicidad de trámites, la dilación en procesos académico-administrativos y desembolsos económicos imprevistos de movilidad y reprografía. En este contexto, la implementación de la presente plataforma web se justifica por la necesidad apremiante de transparentar los flujos informativos y coordinar con eficiencia la interacción entre las dependencias administrativas y el alumnado.

Al consolidar un entorno digital centralizado y verificado, se garantiza la accesibilidad e integridad de la información oficial. Ello permite mitigar el margen de error en la integración y entrega de expedientes, reducir los costos operativos y de traslado para los estudiantes, y optimizar la capacidad operativa del personal universitario al sistematizar la atención de consultas recurrentes.

ALCANCES Y LIMITACIONES

1. Alcance del Proyecto

El desarrollo de la plataforma web abarcará las siguientes dimensiones operativas, funcionales y tecnológicas:

- **Alcance Institucional y de Usuarios:**
    - Plataforma delimitada de manera exclusiva a la comunidad universitaria de la Universidad Tecnológica de Torreón (UTT) (alumnos, docentes, directivos y personal administrativo).
    - Mecanismo de autenticación restringido estrictamente al uso de cuentas con el dominio de correo institucional (@utt.edu.mx).
- **Alcance Funcional (Requisitos Funcionales):**
    - *Gestión de Roles y Accesos:* Implementación de tres niveles de permisos: **Lector** (consulta), **Departamento** (gestión de contenido) y **Administrador** (gestión global de usuarios, departamentos y auditoría).
    - *Muro de Avisos Centralizado:* Módulo de publicación departamental con soporte para archivos adjuntos, búsqueda por palabras clave y filtros multidimensionales (por fecha y departamento).
    - *Módulo de Soporte Directo:* Canal de atención bidireccional mediante mensajería individual entre estudiantes y dependencias administrativas, estructurado en pestañas de mensajes.
    - *Auditoría y Monitoreo de Seguridad:* Registro automatizado de inicio de sesión con captura de dirección IP, fecha, hora y rol, complementado con un control estricto de concurrencia (máximo 2 sesiones activas simultáneas).
- **Alcance Técnico y de Seguridad (Requisitos No Funcionales):**
    - *Comunicación en Tiempo Real:* Transmisión instantánea de mensajes en el chat mediante arquitectura de WebSockets.
    - ***Sistema de Notificaciones:*** **Emisión de notificaciones push globales para entornos móviles y de escritorio mediante Firebase Cloud Messaging (FCM) o Service Workers (PWA). ????**
    - ***Almacenamiento e Interfaz:*** **Gestión de archivos multimedia respaldada en servicios de la nube (AWS S3, Cloudinary o Firebase Storage) con URLs persistentes en PostgreSQL, diseño responsivo adaptativo (*Mobile-First*) y paginación de contenido en el backend.**
    - *Seguridad Aplicativa:* Encriptación de contraseñas mediante hashing (bcrypt), sanitización de datos contra Cross-Site Scripting (XSS), consultas parametrizadas contra inyección SQL (SQLi) y limitación de peticiones (*Rate Limiting*).

2. Limitaciones del Proyecto

Se establecen las siguientes fronteras tecnológicas, operativas y metodológicas que quedan fuera del dominio del presente anteproyecto:

- **Limitaciones de Interoperabilidad e Integración:**
    - *Ausencia de Integración con Control Escolar:* El sistema operará de manera independiente y desacoplada (fase piloto); no se sincronizará ni conectará con la base de datos del sistema de control escolar existente ni con el portal oficial de la UTT.
    - *Carácter No Transaccional:* El sistema es de uso informativo y comunicativo; no procesará transacciones financieras, pagos de inscripción, cobro de trámites ni expedición automatizada de documentos oficiales.
- **Limitaciones Funcionales y de Flujo (RF):**
    - *Restricción de Flujos de Mensajería:* El módulo de chat no permitirá la comunicación entre departamentos ni la iniciación de conversaciones unidireccionales desde las áreas administrativas hacia los alumnos sin una interacción previa iniciada por el usuario Lector.
    - *Tope de Concurrencia:* Acceso restringido a un máximo de 2 sesiones activas por usuario; el sistema invalidará el acceso a cualquier intento posterior.
- **Limitaciones Técnicas y de Infraestructura (RNF):**
    - *Límite y Formato de Archivos:* La carga de archivos adjuntos (en publicaciones y chat) estará acotada a un tamaño máximo de 10 MB por elemento, restringida únicamente a formatos específicos (.pdf, .docx, .jpg, .png).
    - *Dependencia de Notificaciones Push:* La entrega de notificaciones push estará sujeta a la autorización explícita de permisos por parte del usuario en su navegador/dispositivo y a las políticas del servicio de PWA/FCM, omitiendo expresamente el envío de mensajería masiva por SMS o correo electrónico.
- **Limitaciones Metodológicas y de Proyecto:**
    - *Restricción Temporal y Académica:* El tiempo de desarrollo e implementación se ajusta estrictamente al calendario académico del cuatrimestre y a la disponibilidad de horarios del equipo de desarrollo.
    - *Curva de Aprendizaje:* La ejecución contempla tiempos de adaptación a las herramientas tecnológicas, librerías seleccionadas y adopción de metodologías ágiles registradas mediante bitácoras de avance.

PRODUCTO MÍNIMO VIABLE (MVP)

Para garantizar la viabilidad técnica y el cumplimiento del cronograma académico, el desarrollo del sistema se enfocará en la entrega de un Producto Mínimo Viable (MVP).

Este se define como el conjunto esencial de funcionalidades que permiten poner en operación la plataforma "UTT Connect" y validar su impacto en la comunicación institucional.

El MVP contemplará la ejecución integrada del siguiente flujo principal:

1. **Módulo de Autenticación y Roles:** Control de acceso mediante credenciales institucionales (@utt.edu.mx) distinguiendo roles de *Lector*, *Departamento* y *Administrador*.
2. **Gestión Departamental y Publicación:** Panel administrativo para el alta de departamentos y módulo de creación/visualización de avisos oficiales en el muro principal.
3. **Mensajería Bidireccional:** Interfaz de chat en tiempo real (*WebSockets*) habilitada desde el perfil de alumno hacia las cuentas departamentales para la atención de consultas.

Las funcionalidades avanzadas de seguridad perimetral (auditoría de IP, Rate Limiting) y notificaciones push nativas se integrarán de forma incremental tras la validación de la fase MVP.

REQUISITOS FUNCIONALES (RF)

Módulo de Interfaz Pública (Landing Page):

- **RF-01 Landing Page Informativa:** El sistema debe contar con una página de inicio pública que explique de forma concisa el propósito del foro "UTT Connect". Debe incluir un encabezado claro, imágenes descriptivas y llamadas a la acción (CTAs) para iniciar sesión o registrarse.
- **RF-02 Sección de Preguntas Frecuentes (FAQ):** La Landing Page debe incluir una sección de preguntas y respuestas implementada mediante componentes desplegables (acordeones).
- **RF-03 Comportamiento Exclusivo del FAQ:** Al interactuar con la sección de Preguntas Frecuentes, el sistema debe asegurar que solo una respuesta esté visible a la vez. Al abrir un desplegable, cualquier otro que estuviera abierto debe cerrarse automáticamente.

Módulo de Autenticación y Seguridad:

- **RF-04 Autenticación Estricta:** El sistema debe permitir el inicio de sesión utilizando exclusivamente cuentas de correo con el dominio @utt.edu.mx.

- **RF-05 Auto-Registro de Usuarios (Rol Lector):** El sistema debe permitir que los alumnos, profesores y directivos creen su propia cuenta utilizando estrictamente su correo institucional.
- **RF-06 Verificación de Identidad por OTP:** Durante el registro, el sistema enviará un Código de Contraseña de un Solo Uso (OTP) al correo institucional. El usuario debe ingresar este código para verificar su identidad y establecer su contraseña.
- **RF-07 Medidor Visual de Contraseñas:** En los formularios de creación y cambio de contraseña, el sistema mostrará un indicador visual (barra de colores) en tiempo real que evalúe qué tan fuerte es la contraseña basándose en su longitud.
- **RF-08 Recuperación y Cambio de Contraseña (OTP):** El sistema contará con un flujo de "Olvidé mi contraseña" y "Cambiar contraseña". Ambos procesos exigirán la validación obligatoria mediante código OTP enviado al correo registrado.
- **RF-09 Inmutabilidad del Correo Electrónico:** Una vez que un usuario registra y verifica su correo institucional, el sistema bloqueará cualquier intento de modificación de este campo.
- **RF-10 Control de Roles:** El sistema gestionará el acceso mediante 3 roles estrictos: Lector (alumnos, profes, directivos), Departamento y Admin.
- **RF-11 Control de Sesiones Concurrentes:** El sistema debe restringir el acceso a un máximo de 2 sesiones activas simultáneas por usuario. Un tercer intento rechazará el acceso o invalidará la sesión más antigua.
- **RF-12 Auditoría de Sesiones:** El sistema debe registrar de forma automática cada inicio de sesión, capturando el usuario, rol, fecha y hora exacta.
- **RF-13 Registro y Gestión de Direcciones IP:** El módulo de auditoría capturará y registrará la dirección IP exacta de cada inicio de sesión para monitoreo de red.

Módulo de Administración (Rol: Admin):

- **RF-14 Gestión de Departamentos (CRUD):** El administrador debe poder crear, leer, actualizar y eliminar departamentos (Nombre, Descripción y Foto/Logo).
- **RF-15 Gestión de Cuentas de Departamento (CRUD):** El administrador será el único capaz de crear cuentas para los gestores (Nombre, Correo institucional, Contraseña, Foto de perfil) y asignarlos a un Departamento.
- **RF-16 Dashboard de Búsqueda y Filtros:** El panel del administrador contará con vistas de lista para usuarios y departamentos, barras de búsqueda por nombre y filtros de pertenencia.

Módulo del Muro de Avisos (Foro):

- **RF-17 Creación de Publicaciones (Rol: Departamento):** Los departamentos pueden crear avisos (título, texto, adjuntos). El aviso mostrará el Nombre del Departamento y del Usuario creador.
- **RF-18 Edición y Eliminación de Avisos:** Un departamento solo podrá editar o eliminar sus propias publicaciones.
- **RF-19 Visualización y Filtrado de Avisos (Lector y Departamento):** El feed principal incluirá botones de filtro por departamento, filtro por fecha y barra de búsqueda por palabras clave.

Módulo de Soporte Directo (Chat):

- **RF-20 Inicio de Conversaciones:** Solo el rol Lector puede iniciar chat con un Departamento. Los departamentos NO inician chats con otros departamentos.
- **RF-21 Intercambio de Mensajes y Archivos:** El chat permitirá envío bidireccional de texto y archivos adjuntos (imágenes, PDF, DOCX).
- **RF-22 Gestión de Bandeja de Entrada:** La vista de mensajes mostrará pestañas de "Activos" y "Archivados", con notificaciones visuales (badges/globos) de mensajes no leídos.

Módulo de Notificaciones:

- **RF-23 Notificaciones Push Globales:** El sistema enviará una notificación push (PC, Laptop, Celular) a todos los usuarios cuando se publique un nuevo aviso.

REQUISITOS NO FUNCIONALES (RNF)

Módulo de Interfaz y Experiencia de Usuario (UI/UX):

- **RNF-01 Diseño Responsivo Integral (Mobile-First):** Toda la interfaz web (Landing, dashboard, foro, chat) debe adaptarse fluidamente a móviles, tablets y escritorios (usando menús inferiores en móviles y laterales en escritorio).
- **RNF-02 Arquitectura y SEO de la Landing Page:** La Landing Page debe desarrollarse siguiendo las guías de estructura semántica y rendimiento de web.dev (Core Web Vitals).
- **RNF-03 Inyección de Scripts y Manejo del DOM (Angular):** Al usar Angular, el manejo del DOM (ej. acordeones) se hará vía directivas del framework. Se prohíbe usar etiquetas <script> al final del <body> a menos que sea para bibliotecas externas estrictamente necesarias.

Módulo de Arquitectura y Rendimiento:

- **RNF-04 Arquitectura en Tiempo Real:** El chat se implementará con WebSockets (ej. Socket.io) para entrega de mensajes sin recargar la página.
- **RNF-05 Rendimiento de Carga (Paginación):** El backend implementará paginación (scroll infinito o carga por bloques) en el muro de avisos.
- **RNF-06 Gestión y Límite de Archivos:** El sistema limitará la subida de adjuntos a 10 MB por archivo (formatos permitidos: PDF, DOCX, JPG, PNG).
- **RNF-07 Almacenamiento en la Nube:** Las imágenes y PDFs se almacenarán en servicios cloud (ej. AWS S3, Firebase Storage), guardando solo la URL en la base de datos (PostgreSQL).
- **RNF-08 Servicio de Notificaciones Push:** Se implementará Firebase Cloud Messaging (FCM) o PWA Service Workers para las notificaciones nativas.

Módulo de Ciberseguridad y Privacidad:

- **RNF-09 Estándares de Contraseñas (NIST 800-63B):** Las contraseñas exigirán un mínimo de 8 caracteres, recomendarán 15+ y tendrán un máximo de 64, priorizando la longitud sobre reglas arbitrarias de caracteres especiales.
- **RNF-10 Seguridad de Contraseñas (Hashing):** Toda contraseña será encriptada (ej. bcrypt) antes de guardarse en la base de datos.
- **RNF-11 Prevención de Inyección SQL (SQLi):** Uso estricto de consultas parametrizadas o un ORM en el backend.
- **RNF-12 Protección contra Cross-Site Scripting (XSS):** Sanitización estricta de entradas y salidas de texto para bloquear ejecución de scripts maliciosos en el cliente.
- **RNF-13 Rate Limiting (Fuerza Bruta):** El backend limitará la cantidad de peticiones en los endpoints de login y validación OTP, bloqueando temporalmente la IP tras intentos fallidos.

ANÁLISIS DE RIESGOS

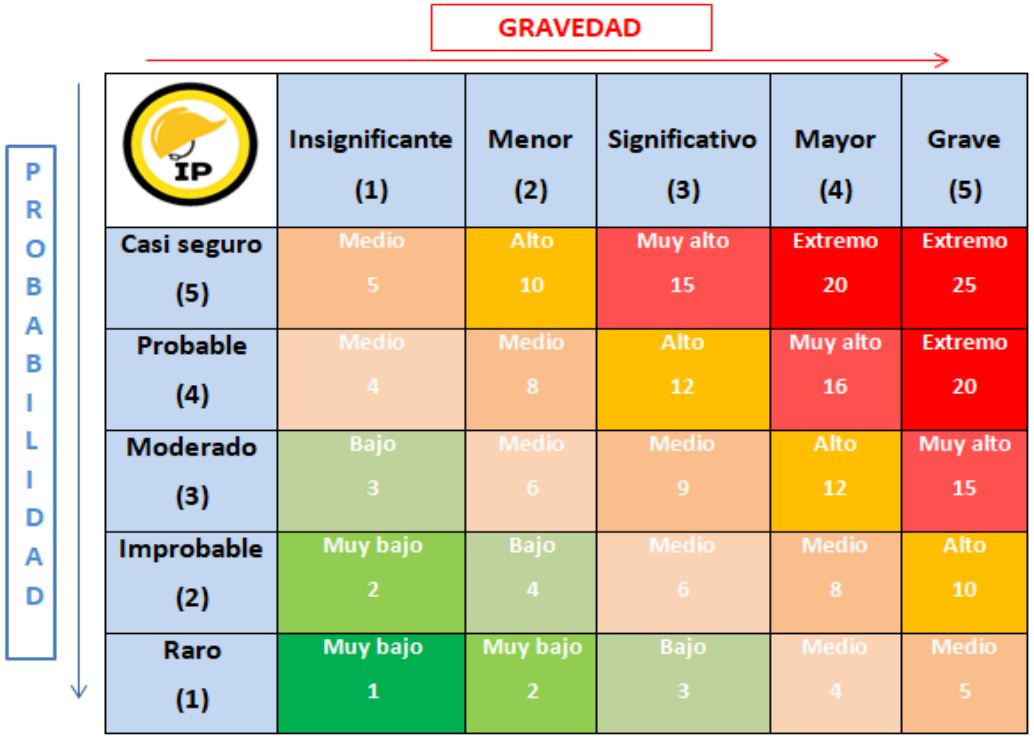
ID	RIESGO	CONSECUENCIA	VALOR	RESPUESTA	VALORACIÓN
R01	Fallo o retraso en el envío de códigos OTP vía correo institucional.	Los usuarios no pueden completar el auto-registro, verificar su identidad ni recuperar sus contraseñas.	**20**	Implementar colas de envío asíncrono (ej. BullMQ/Redis) con reintentos automáticos, vigencia de código a 10 minutos y habilitar botón de reenvío tras 60 segundos.	Crítico (Alto)
R02	Pérdida de conexión o latencia en la arquitectura WebSocket.	Los mensajes del chat departamental no se entregan en tiempo real, requiriendo recargar la página.	**20**	Configurar reconexión automática con *fallback* a HTTP Polling en Socket.io y almacenar mensajes no enviados en la sesión local (IndexedDB) hasta reconectar.	Crítico (Alto)
R03	Bloqueo o rechazo de Notificaciones Push mediante FCM / Service Workers.	La comunidad universitaria no recibe avisos institucionales en PC o móviles de forma nativa.	**15**	Integrar un centro de notificaciones *in-app* (módulo interno con badge de no leídos) para consultar alertas directamente dentro del sistema si el navegador bloquea las push.	Medio
R04	Agotamiento del almacenamiento en la nube o exceso del límite de 10	Rechazo en la carga de adjuntos (PDF/imágenes) en avisos y chat, interrumpiendo la	**20**	Validar formato y peso estrictamente en frontend/backend, aplicar compresión automática de imágenes antes de la subida y	Crítico (Alto)

	MB por archivo.	gestión de trámites.		activar alertas de consumo en el servicio cloud.	
R05	Vulnerabilidades de Inyección SQL (SQLi) o Cross-Site Scripting (XSS)	Compromiso de la base de datos PostgreSQL, manipulación de búsquedas o ejecución de scripts maliciosos.	**25**	Utilizar consultas parametrizadas mediante un ORM (TypeORM/Prisma), sanitizar entradas de texto enriquecido con DOMPurify y configurar políticas CSP en el servidor.	Crítico (Alto)
R06	Bloqueo accidental de usuarios por Rate Limiting en IP compartida.	Múltiples alumnos intentando iniciar sesión desde la red WiFi de la UTT son bloqueados temporalmente.	**16**	Configurar el *Rate Limiting* evaluando de forma combinada la dirección IP junto con el identificador/correo del usuario, y agregar la subred institucional a una lista blanca.	Alto
R07	Incumplimiento de estándares NIST 800-63B o hashing inseguro de contraseñas.	Aceptación de claves vulnerables a ataques de fuerza bruta o almacenamiento expuesto en BD.	**20**	Aplicar la regla de longitud de 8 a 64 caracteres en formularios, encriptar claves con `bcrypt` (factor de costo ≥ 10) e integrar el medidor visual de fortaleza en tiempo real.	Crítico (Alto)
R08	Incompatibilidad de la interfaz responsiva (*Mobile-First*)	Elementos del chat o la *Bottom Navigation Bar* inalcanzables o desalineados en pantallas móviles.	**12**	Aplicar diseño con reglas CSS flexbox/grid y realizar pruebas de usabilidad multi-dispositivo y emuladores para validar la barra inferior en móviles y lateral en escritorio.	Medio
R09	Carga lenta del feed por fallas en la paginación o	Congelamiento o alto consumo de memoria en el navegador al acumular cientos de	**12**	Implementar paginación basada en cursores (*limit/offset*) en el backend e indexar los campos de búsqueda por fecha y	Medio

	scroll infinito.	publicaciones.		departamento en PostgreSQL.	
R10	Error en la manipulación del DOM mediante directivas de Angular o uso de scripts externos.	Fallas de renderizado en componentes de la Landing Page (acordeón FAQ) y problemas de seguridad frontend.	**10**	Construir componentes dinámicos (acordeones y modales) utilizando exclusivamente directivas e inyección de dependencias de Angular, evitando librerías JS manipular el DOM directamente.	Medio
R11	Fuga de permisos en la matriz de roles (RBAC).	Un usuario departamental edita o elimina avisos de otra área o accede al panel del Administrador.	**20**	Implementar *Guards* en Angular y middlewares de autorización en el backend que verifiquen el Token JWT, el rol y el `departamento_id` antes de ejecutar cualquier operación CRUD.	Crítico (Alto)
R12	Intentos de modificación fraudulenta del correo institucional registrado.	Vinculación no autorizada de cuentas a correos externos ajenos al dominio `@utt.edu.mx`.	**20**	Bloquear el atributo correo como campo inmutable tanto en la capa de modelo/DTO de actualización como mediante *triggers* de restricción en PostgreSQL.	Crítico (Alto)
R13	Inconsistencia en el registro de direcciones IP y auditoría de sesiones	Registro ineficaz de auditoría al no detectar la IP real detrás de proxies o redes institucionales.	**8**	Configurar la lectura de la cabecera `X-Forwarded-For` en el servidor backend (Nginx/Node.js) para extraer la IP pública/privada cliente real antes de guardarla en la tabla de auditoría.	Bajo

R14	Acumulación de consultas no atendidas en la bandeja departamental	Demora en la resolución de dudas del alumnado y saturación de la vista de chats activos.	**15**	Diseñar pestañas de organización (Activos/Archivados), contadores visuales (*badges*) de no leídos y respuestas automáticas fuera de horario de atención.	Medio

EJEMPLO MATRIZ DE ANÁLISIS DE RIESGOS (5x5):



Fuente: Informacionprevencion

METODOLOGÍA DE DESARROLLO

Marco Metodológico: Enfoque Híbrido Scrumban (Scrum + Kanban)

Para la gestión, diseño y construcción de la plataforma web, se implementará una metodología ágil de carácter híbrido conocida como Scrumban, la cual combina la estructura temporal y orientada a metas de Scrum con la visibilidad y optimización del flujo de trabajo continuo que ofrece Kanban. Esta integración permite un seguimiento

riguroso y sistemático de las etapas del ciclo de vida del software, adaptándose eficientemente a los tiempos del cronograma académico.

1. Implementación del Marco Scrum (Estructura Incremental y Temporal)

El núcleo de la planificación del proyecto se basará en la división del desarrollo en ciclos iterativos de tiempo fijo denominados **Sprints** (con una duración definida de entre una y dos semanas). Cada Sprint estará enfocado en la entrega de incrementos funcionales y evaluables del sistema (módulos probados y operativos).

- Delimitación de Roles:
    - **Product Owner / Gestor de Requisitos:** Define y prioriza el *Product Backlog* en función de las necesidades institucionales y los Requisitos Funcionales (RF) y No Funcionales (RNF) establecidos.
    - **Scrum Master / Facilitador:** Supervisa el cumplimiento de los tiempos, mitiga los bloqueos técnicos reportados en la matriz de riesgos y garantiza la adherencia a la metodología.
    - **Equipo de Desarrollo (*Development Team*):** Encargado de la arquitectura, diseño UI/UX, programación *frontend/backend*, integración de servicios en la nube y pruebas de software.
- Ceremonias del Ciclo de Vida:
    - **Planificación del Sprint (*Sprint Planning*):** Definición de los objetivos inmediatos a cumplir y selección de tareas del *Backlog*.
    - **Seguimiento Continuo (*Daily Stand-up*):** Revisiones breves de avance individual para detectar y solventar posibles cuellos de botella de forma oportuna.
    - **Revisión y Retrospectiva del Sprint (*Sprint Review & Retrospective*):** Evaluación del incremento funcional entregado (módulos de autenticación, muro, chat, etc.) y ajuste de dinámicas de trabajo para el siguiente ciclo.

2. Implementación de Kanban (Control Visual del Flujo de Trabajo)

Para garantizar la transparencia operativa y evitar la saturación en la capacidad de desarrollo, se integrará un **Tablero Kanban** digital como herramienta de control visual. Este tablero permitirá monitorear el estado exacto de cada historia de usuario y tarea técnica durante el transcurso de los *Sprints*.

- Estados del Flujo de Trabajo (Columnas del Tablero):
    1. **Por Hacer (*Backlog del Sprint*):** Tareas priorizadas asignadas al ciclo actual que aún no han comenzado su ejecución.

2. **En Progreso (*In Progress*):** Actividades activamente en desarrollo (código backend, maquetación frontend, integración de APIs).
3. **En Revisión / Control de Calidad (*In Review / QA*):** Tareas finalizadas en etapa de pruebas unitarias, sanitización de seguridad (XSS/SQLi) y validación de diseño responsivo.
4. **Completado (*Done*):** Funcionalidades completamente integradas, probadas e implementadas en el entorno de despliegue.

- Gestión de Limites de Trabajo (WIP - Work in Progress): Se restringirá el número de tareas simultáneas en la columna En Progreso para priorizar la culminación de funcionalidades críticas sobre la apertura de nuevos requerimientos.

3. Sinergia y Justificación de la Metodología

La convergencia de Scrum y Kanban en el desarrollo de la plataforma ofrece las siguientes ventajas operativas:

- Trazabilidad y transparencia: Permite visualizar en todo momento qué integrante está trabajando en qué componente específico y cuál es el estado general del proyecto.
- Mitigación de Riesgos: Al realizar entregas parciales y continuas mediante *Sprints*, los errores de arquitectura o seguridad se detectan de forma temprana antes del despliegue final.
- Flexibilidad Controlada: Combina la rigidez necesaria para cumplir los plazos académicos institucionales con la flexibilidad para ajustar prioridades operativas ante imprevistos técnicos.

ROLES DE LOS INTEGRANTES

Integrante	Rol Principal	Áreas de Desarrollo
Aníbal	Scrum Master & QA Lead	Desarrollo Fullstack, Aseguramiento de Calidad (QA) y Administración de BD (DBA)
Meredith	UX/UI Lead Designer	Diseño de Interfaces/Experiencia de Usuario, Desarrollo Fullstack

		y Administración de BD (DBA)
Ernesto	Fullstack Developer	Desarrollo Backend/Frontend y Administración de BD (DBA)
César	UX/UI Designer	Diseño de Interfaces/Experiencia de Usuario, Desarrollo Fullstack y Administración de BD (DBA)
Sara	Product Owner & Product Manager	Gestión del Producto, Desarrollo Fullstack y Administración de BD (DBA)

TAREAS DEL PROYECTO

Muro de Avisos y Roles (Octubre 16 - 31)

Tarea	Asignado	Esfuerzo	Dependencias	Criterios de Terminado (DoD)
Diseño UI/UX: Dashboard Admin y Muro de Avisos	Meredith, César	8 hrs	Ninguna	Vistas aprobadas por PO (Sara), mockups exportables en Figma, componentes listos para Angular.

Backend: CRUD Departamentos, Usuarios y Guards RBAC	Ernesto, Aníbal	24 hrs	Diseño de BD (Fase 1)	Endpoints probados en Postman, contraseñas hasheadas (bcrypt), tokens JWT funcionales, restricción de rutas por rol implementada.
Frontend: Dashboard Admin y Gestión de Usuarios	César	20 hrs	UI/UX, Backend Auth	Vistas construidas en Angular, formularios reactivos con validación, conexión con API exitosa, manejo de errores visible al usuario.

Backend: CRUD Avisos, Paginación y Subida a Nube (S3/Firebase)	Sara, Ernesto	24 hrs	Backend Auth	Paginación funcional (máx. 10 por petición), archivos multimedia guardan URL correcta en PostgreSQL, validación de límite de 10MB.
Frontend: Muro de Avisos, Filtros y Formulario de Carga	Meredith	20 hrs	UI/UX, Backend CRUD Avisos	Scroll infinito o paginación en UI, filtros operativos (fecha/departamento), previsualización de adjuntos antes de publicar.
QA Sprint 2	Aníbal	8 hrs	Todas las anteriores	Pruebas E2E superadas, prevención de XSS/SQLi verificada, 0 bugs críticos.

Chat y Notificaciones (Noviembre 1 - 15)

Tarea	Asignado	Esfuerzo	Dependencias	Criterios de Terminado (DoD)
Diseño UI/UX: Interfaz de Chat y Bandeja	César	6 hrs	Ninguna	Flujo de mensajería (mobile-first), diseño de notificaciones y estados de lectura aprobados.
Backend: Config. WebSockets (Socket.io) y Salas de Chat	Ernesto	30 hrs	Backend Auth (Sprint 2)	Conexión WS estable, aislamiento de salas por usuario/departamento, almacenamiento de historial de chat en BD.

Frontend: Interfaz de Chat (Mobile-First) y Cliente Socket.io	Meredith, César	25 hrs	UI Chat, Backend WS	Mensajes se envían y reciben sin recargar la página, diseño adaptativo a móviles comprobado.
Backend: Notificaciones Push (FCM) y Estado de Mensajes	Aníbal	15 hrs	Backend WS	Lógica de envío de alertas disparada correctamente al publicar un aviso, actualización de `is_read` en mensajes.

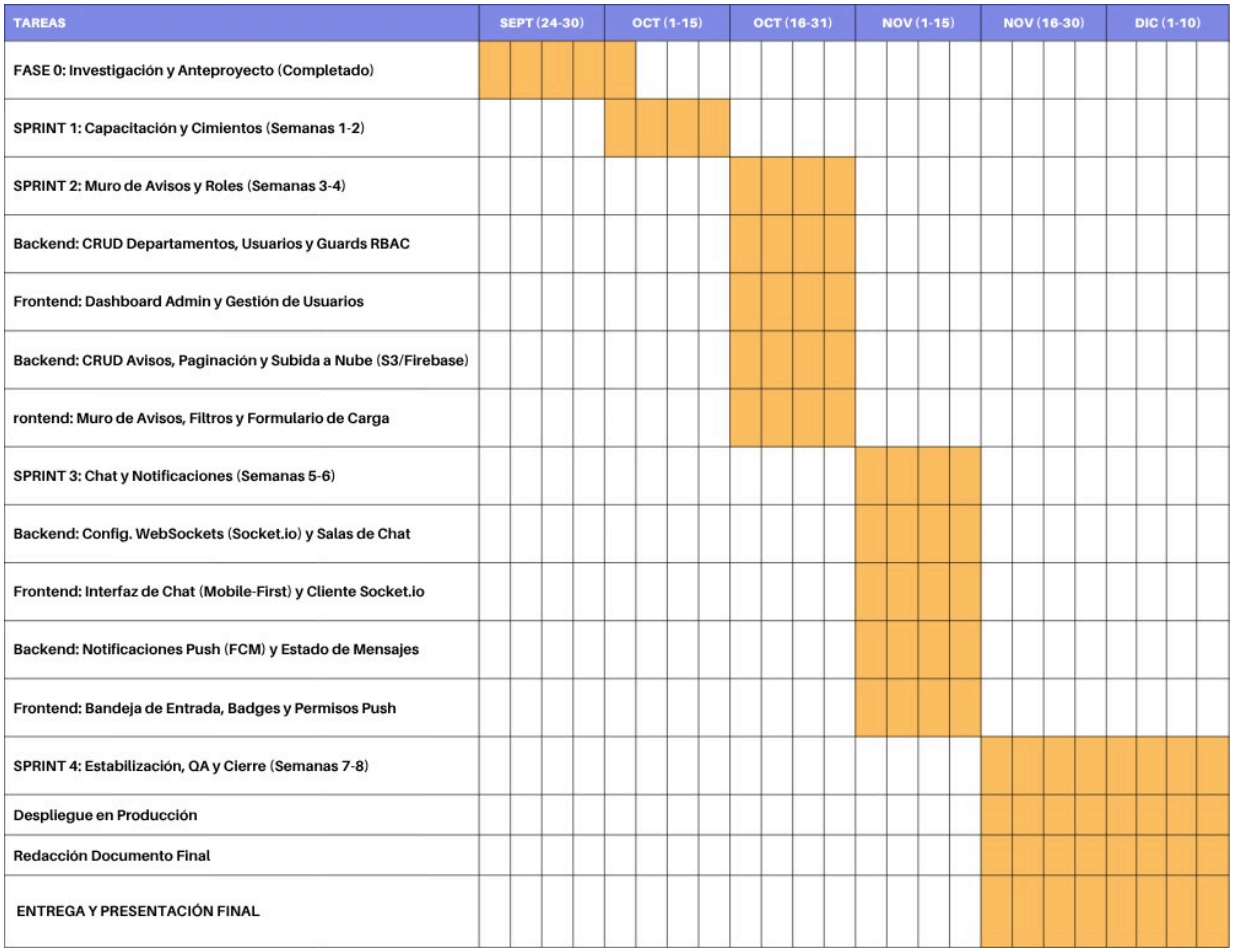
Frontend: Bandeja de Entrada, Badges y Permisos Push	Sara	15 hrs	Backend Push	Pestañas Activos/Archivados funcionales, contador de no leídos exacto, solicitud nativa de permisos push en navegador.
QA Sprint 3	Aníbal	10 hrs	Todas las anteriores	Estrés de WebSockets verificado, notificaciones llegan a dispositivos prueba, historial de chat persistente.

Estabilización, QA y Cierre (Noviembre 16 - Diciembre 10)

Tarea	Asignado	Esfuerzo	Dependencias	Criterios de Terminado (DoD)
Auditoría de Seguridad y Corrección de Bugs	Aníbal, Ernesto	20 hrs	Sprint 2 y 3 completos	Resolución de hallazgos del Sprint 2 y 3, verificación de límite de concurrencia de sesiones, código limpio de *console.logs*.
Despliegue en Producción	Ernesto, Aníbal	15 hrs	QA Final aprobado	Servidor configurado, base de datos migrada a entorno de producción, dominio conectado, HTTPS (SSL) activo.
Redacción Documento Final y	Sara, Meredith	20 hrs	Funcionalidad congelada	Documentación técnica completa (RF/RNF actualizados), manual de usuario redactado,

Manuales				diagramas anexados.
Preparación para Entrega y Presentación	Todo el equipo	10 hrs	Despliegue en Producción	Ensayo de demostración en vivo (demo fluida), presentación visual de apoyo terminada.

CRONOGRAMA



ESTRUCTURA Y ARQUITECTURA DEL SISTEMA

Para garantizar la escalabilidad, mantenibilidad y desacoplamiento de componentes de la plataforma web, el desarrollo se fundamentará en un patrón de **Arquitectura en Tres Capas** (*Three-Tier Architecture*), integrado con el modelo de **Aplicación de Página Única** (*Single Page Application - SPA*) en el cliente. Esta estructura permite separar de forma clara la interfaz de usuario, las reglas operativas y el almacenamiento persistente de datos.

1. Arquitectura en Tres Capas

- Capa de Presentación (Cliente / Frontend — Angular): Constituye la interfaz gráfica con la que interactúan los distintos perfiles de la comunidad universitaria. Es la encargada de la renderización dinámica de los avisos en el muro, la gestión del tablero de Preguntas Frecuentes (FAQ), la navegación por perfiles y la interfaz interactiva de mensajería instantánea. Se encarga de la captura de eventos,

validaciones en el cliente y la presentación fluida de los datos consumidos desde el servidor.

- Capa de Lógica de Negocio (Servidor / Backend — NestJS): Actúa como el motor central del sistema. Implementa y ejecuta las reglas del dominio universitario, incluyendo el módulo de autenticación (generación de tokens JWT y validación mediante códigos OTP), la aplicación del control de acceso basado en roles (RBAC: Lector, Departamento y Admin), la gestión del flujo CRUD de avisos y departamentos, la orquestación del servidor de *WebSockets* para el chat en tiempo real y la conexión con servicios en la nube para el almacenamiento multimedia.

- Capa de Persistencia de Datos (Base de Datos — PostgreSQL): Provee un almacenamiento relacional estructurado y seguro. Modela y resguarda la información crítica del sistema, incluyendo esquemas para usuarios, departamentos, publicaciones, historial de conversaciones, tablas de auditoría de sesiones y direcciones IP, garantizando la integridad referencial y la eficiencia en las peticiones mediante consultas parametrizadas e indexación.

2. Patrón de Aplicación de Página Única (Single Page Application — SPA)

La capa de presentación implementada con **Angular** adopta el modelo SPA, lo que transforma la experiencia de uso dentro del portal:

- Navegación Dinámica e Ininterrumpida: Tras la carga inicial del documento HTML base, los desplazamientos entre secciones (del muro de publicaciones al módulo de chat o al panel administrativo) se realizan mediante el enrutador del marco (Angular Router) de forma asíncrona, eliminando las recargas completas del navegador.

- Eficiencia en la Transferencia de Datos: El cliente se comunica con el servidor backend únicamente para intercambiar datos en formato JSON mediante peticiones HTTP/RESTful y canales bidireccionales en tiempo real (*WebSockets*), reduciendo drásticamente el consumo de ancho de banda y acelerando los tiempos de respuesta para el usuario.

TECNOLOGÍAS

Base de datos: PostgreSQL
Se utilizará como sistema gestor de base de datos debido a su estabilidad, seguridad y capacidad para manejar grandes cantidades de información de manera eficiente.

Backend: Nest.JS
Se empleará para el desarrollo del backend, ya que proporciona una estructura organizada y escalable, facilitando la creación y mantenimiento de APIs y servicios.

Frontend: Angular
Será utilizado para el frontend porque permite desarrollar interfaces dinámicas, organizadas y escalables, además de facilitar la creación de aplicaciones web de una sola página.

DIseño/Maquetado: Figma
Se utilizará para el diseño y maquetado de las interfaces, permitiendo crear prototipos, definir la distribución visual y comprobar la experiencia de usuario antes de implementar el sistema.

Diagramas UML: Mermaid
Se empleará para la elaboración de diagramas UML y otros diagramas técnicos, facilitando la representación visual de la estructura, procesos y relaciones del sistema.

Modelado: Autodesk MAYA
Se utilizará para el modelado tridimensional, permitiendo crear y representar elementos 3D que puedan complementar los recursos visuales del proyecto.

DESCRIPCIÓN DE CONTENIDOS (WIREFRAME)

Tabla 1: Inventario de Recursos Textuales (Titulares, Microcopys y Formularios)

Elemento / Componente	Ubicación / Pantalla	Contenido Exacto / Microcopy	Función en la Interfaz
Encabezado Principal	Landing Page	*"Tu universidad, más cerca y mejor conectada.*	Presentar la propuesta de valor de la plataforma.
Subtítulo Informativo	Landing Page	*"El punto de encuentro oficial entre la comunidad universitaria y tus departamentos."*	Explicar el propósito institucional de la plataforma.
Insignia Institucional	Landing Page	*"PLATAFORMA OFICIAL UTT.*	Generar confianza e identidad institucional.
Botones de Acción (CTAs)	Landing Page	*"Iniciar sesión >"* y *"Registrarse"*	Dirigir a los flujos de acceso de usuario.
Microcopy de Restricción	Formulario de Registro	*"Acceso exclusivo con correo @utt.edu.mx"*	Indicar el requisito obligatorio de correo institucional.

Microcopy de Advertencia	Formulario de Registro	*"Tu correo quedará vinculado y no podrá modificarse después de verificarlo"*	Prevenir errores en la captura del correo antes de la validación.
Criterio de Contraseña	Formulario de Registro	*"Min. 8 - recomendado 15+"*	Orientar sobre el nivel de seguridad de la clave.
Botón de Envío	Formulario de Registro	*"Enviar código de verificación"*	Procesar el registro y detonar el envío del código OTP.
Preguntas Frecuentes (FAQ)	Sección FAQ (Acordeón)	*"¿Quién puede usar UTT Connect?", "¿Los avisos son oficiales?", "¿Cómo contacto a un departamento?", "¿Cómo se protegen mis datos?"*	Resolver dudas operativas recurrentes de la comunidad.
Respuesta de FAQ	Sección FAQ (Desplegable)	*"Estudiantes, docentes, directivos y gestores con una cuenta institucional activa @utt.edu.mx."*	Explicar el alcance y perfiles de usuarios.

Tarjeta de Publicación	Muro de Avisos (Feed)	*"Coordinación de Becas - Convocatoria Becas de Manutención 2026"*	Difundir avisos y convocatorias académicas.

Tabla 2: Inventario de Recursos Visuales e Iconografía

Recurso Visual	Elemento / Componente	Descripción y Aplicación	Ubicación en la Interfaz
Logotipo Oficial	Identificador UTT Connect	Gráfico vectorial de la marca institucional con isotipo de conexión.	Encabezado superior (*Header*) en todas las pantallas.
Icono de Notificación	Campana de Alertas	Icono vectorial para indicar avisos o notificaciones no leídas.	Barra superior de navegación.
Icono de Perfil	Silueta de Usuario	Ilustración de perfil para acceso y formulario de registro.	Encabezado del formulario de registro y perfil.
Icono de Despliegue	Chevron (*Flecha abajo/arriba*)	Control visual para expandir o contraer respuestas.	Componente de acordeón en la sección FAQ.
Casilla de Selección	Checkbox de Términos	Elemento interactivo para confirmación de términos.	Pie del formulario de registro.

Tabla 3: Fuentes Tipográficas y Fuentes de Información

Tipo de Fuente	Categoría / Nombre	Especificaciones Técnicas	Justificación / Origen
Tipografía Principal	*Inter* / *Roboto* (Sans-serif)	Pesos: Bold (700) para títulos, Medium (500) para botones y Regular (400) para cuerpo de texto.	Diseñada para interfaces web, optimizada para lectura en pantallas móviles y de escritorio.
Fuentes de Información	Coordinaciones y Departamentos UTT	Publicaciones, convocatorias, fechas límite y avisos institucionales emitidos por cada área.	Información verificada y proporcionada directamente por las dependencias de la UTT.
Marco Legal	Normativa Institucional UTT	Contenido de los términos de uso y el aviso de privacidad de la plataforma.	Garantiza el cumplimiento de la política de protección de datos institucionales.

Tabla 4: Tono de Comunicación y Criterios de Selección

Dimensión	Criterio Aplicado	Explicación y Justificación
Tono Institucional y Accesible	Lenguaje Formal e Incluyente	Estilo neutro y respetuoso adecuado para un entorno universitario, facilitando la comprensión para alumnos de todos los cuatrimestres.
Tono Directo y Guiado	Microcopys Instruccionales	Indicaciones breves en botones e inputs que guían la interacción del usuario sin ambigüedades (ej. *"Min. 8 - recomendado 15+"*).
Tono Informativo y Seguro	Transparencia Operativa	Mensajes preventivos claros sobre el uso exclusivo de correos `@utt.edu.mx` y la oficialidad de las publicaciones.
Criterio Mobile-First	Claridad en Pantallas Pequeñas	Selección de textos breves y botones con área amplia de contacto para facilitar la navegación táctil.
Criterio de Jerarquía	Priorización Visual	Colocación estratégica de llamados a la acción (*CTAs*) y avisos prioritarios en la zona superior de la interfaz.

Tabla 5: Matriz de Licencias y Derechos de Uso

Recurso / Insumo	Tipo de Licencia / Legalidad	Criterio de Uso y Acreditación
Marca e Identidad UTT	Propiedad Intelectual Institucional	El nombre *"UTT Connect"*, logotipos y nombres de departamentos pertenecen a la Universidad Tecnológica de Torreón y se emplean bajo autorización institucional.
Fuentes Tipográficas	SIL Open Font License (OFL)	Fuentes distribuidas a través de Google Fonts, libres de regalías para integración en desarrollo web.
Iconografía e Ilustraciones	Apache License 2.0 / Open Source	Iconos de *Google Material Symbols*, libres de derechos para uso en proyectos informáticos.

JUSTIFICACIÓN DE LA HERRAMIENTA Y PLATAFORMA DE VERSIONAMIENTO
Se ha seleccionado el flujo de trabajo **GitHub Flow** debido a su arquitectura ágil y eficiente, la cual simplifica el control de versiones al operar mediante una única rama principal estable (`main`) y ramas auxiliares de vida corta, reduciendo la sobrecarga administrativa y el riesgo de conflictos complejos de integración (*merge conflicts*). Asimismo, esta metodología promueve las buenas prácticas de desarrollo al alinearse con procesos de Integración y Despliegue Continuo (CI/CD) y exigir el uso obligatorio de solicitudes de extracción (*Pull Requests*); esto garantiza que cada cambio sea sometido a pruebas automatizadas y a una revisión rigurosa de código (*Code Review*) antes de su fusión, asegurando así la trazabilidad, la calidad técnica y la estabilidad constante del sistema.

DEFINICIÓN Y DISEÑO DEL FLUJO DE TRABAJO (WORKFLOW)

Para la gestión de código y la colaboración del proyecto, se ha establecido la implementación del modelo de flujo de trabajo GitHub Flow. Este modelo se basa en una arquitectura ágil y lineal centrada en una única rama principal estable y ramas auxiliares de vida corta.

1. Estructura de Ramas (*Branching Strategy*)
El desarrollo se organizará bajo la siguiente estructura de ramas:

- `main`: Rama principal que representa el código en estado de producción o listo para despliegue. Queda estrictamente prohibido realizar *pushes* directos a esta rama.
- **Ramas de características (*Feature Branches*)**: Ramas temporales creadas a partir de `main` para el desarrollo de nuevas funcionalidades, refactorizaciones o correcciones. Su ciclo de vida concluye al ser fusionadas e integradas mediante un *Pull Request*.

2. Nomenclatura Estándar de Ramas
Para mantener la coherencia y la trazabilidad en el repositorio, las ramas temporales deberán nombrarse utilizando los siguientes prefijos en minúsculas, seguidos de una breve descripción en *kebab-case*:

- `feature/`: Desarrollo de nuevos módulos o funcionalidades (ej. `feature/autenticacion-jwt`).
- `fix/` o `bugfix/`: Corrección de errores en etapa de desarrollo o pruebas (ej. `fix/error-cors`).
- `hotfix/`: Soluciones urgentes destinadas a la rama principal (ej. `hotfix/fallo-pago`).
- `refactor/`: Reestructuración u optimización de código existente sin alterar su comportamiento funcional (ej. `refactor/servicio-usuario`).

- docs/: Modificaciones exclusivas a la documentación del proyecto (ej. docs/actualizacion-readme).

3. Convención de Commits (*Conventional Commits*)
Los mensajes de *commit* deberán seguir el estándar de *Conventional Commits* para facilitar la lectura del historial de cambios: <tipo>(<alcance>): <descripción corta en presente>.

- feat: Incorporación de una nueva funcionalidad.
-
- fix: Solución de un defecto o bug.
-
- refactor: Cambios en el código que no corrigen bugs ni añaden funcionalidades.
-
- docs: Cambios en archivos de documentación.
-
- style: Ajustes de formato, espacios o punto y coma (sin cambios en la lógica).
-
- test: Adición o corrección de pruebas unitarias o de integración.
-

Ejemplo: feat(auth): agregar middleware para validacion de tokens JWT

4. Políticas de Combinación y Solicitudes de Extracción (*Pull Requests*)
La integración de código a la rama main se regirá bajo las siguientes políticas de calidad y seguridad:

- **Aprobación de Revisiones (*Code Review*):** Todo *Pull Request* deberá contar con al menos una aprobación obligatoria por parte del equipo antes de su fusión.
-
- **Verificación de Pruebas Automatizadas:** No se permitirá la combinación de código si los flujos de Integración Continua (CI) mediante *GitHub Actions* no concluyen exitosamente.
-
- **Estrategia de Combinación (*Squash and Merge*):** Los *commits* pertenecientes a una rama de característica se condensarán en un único *commit* representativo al momento de la fusión, manteniendo un historial limpio y lineal en la rama main.

Configuración y Parámetros del Entorno de Versiones

Para garantizar la integridad, la seguridad y la correcta administración del repositorio del proyecto en GitHub, se ha establecido la configuración inicial del entorno de control de versiones bajo los siguientes lineamientos:

1. Configuración de Parámetros Iniciales

El entorno local de Git y la plataforma remota de GitHub han sido inicializados bajo parámetros estandarizados:

- **Rama predeterminada:** Se define la rama `master` como la rama principal y única para código en estado de producción.

- **Identificación de colaboradores:** Todos los integrantes del equipo cuentan con su identidad global configurada (`user.name` y `user.email`) coincidente con sus cuentas de GitHub para asegurar la correcta atribución y trazabilidad de los *commits*.

git config --global user.name "Nombre Completo"
git config --global user.email "correo@ejemplo.com"

2. Control de Accesos y Permisos

La gestión del acceso al repositorio se administra bajo el principio de menor privilegio mediante las herramientas de organización y colaboradores de GitHub:

- **Roles de usuario:** Se asignan permisos de escritura (*Write*) a los desarrolladores del proyecto para la creación de ramas secundarias y apertura de *Pull Requests*, restringiendo la administración directa de la configuración del repositorio a roles de mantenimiento (*Admin*).
-
- **Protección de ramas (*Branch Protection Rules*):** La rama `main` cuenta con reglas de protección estrictas que impiden la sobrescritura del historial (*force push*), la eliminación de la rama y los envíos directos sin una revisión de código preaprobada.
-

3. Exclusión de Archivos y Limpieza (`.gitignore`)

Para evitar la contaminación del repositorio con archivos innecesarios o sensibles, se ha implementado un archivo `.gitignore` estandarizado según la pila tecnológica del proyecto. Se excluyen explícitamente:

- Dependencias y paquetes de terceros (ej. `node_modules/`, `vendor/`).
-
- Archivos de compilación y artefactos generados (ej. `dist/`, `build/`).
-
- Archivos de configuración del entorno de desarrollo (ej. `.vscode/`, `.idea/`).
-
- Archivos del sistema operativo (ej. `.DS_Store`, `Thumbs.db`).
-
- Archivos de variables de entorno y credenciales locales (ej. `.env`, `.env.local`).

4. Credenciales de Seguridad y Autenticación
Con el propósito de proteger la infraestructura y los recursos del proyecto, se aplican las siguientes medidas de seguridad:

- **Autenticación SSH y Personal Access Tokens (PAT):** La interacción entre las máquinas locales y el repositorio remoto de GitHub se realiza exclusivamente mediante llaves cifradas **SSH** y tokens de acceso personal con permisos acotados, descartando el uso de contraseñas convencionales.
-
- **Gestión de Variables de Entorno y Secretos:** Queda estrictamente prohibida la inclusión de llaves API, contraseñas de bases de datos o tokens de autenticación dentro del código fuente. Los datos de configuración sensibles se gestionan mediante archivos de entorno locales excluidos del control de versiones y a través de **GitHub Actions Secrets** para los flujos automatizados de Integración Continua (CI/CD).

REPOSITORIO EN FUNCIONAMIENTO Y EVIDENCIAS
URL del repositorio del FRONTEND:
https://github.com/UTTCONNECT/utt_connect-frontend.git
URL del repositorio del BACKEND:
https://github.com/UTTCONNECT/utt_connect-backend.git

Ambos repositorios contienen la evidencia de la colaboración, reflejada en el historial de commits estructurados, fusiones exitosas de ramas y revisiones a través de Pull Requests. Además, el archivo README.md alojado en la raíz del proyecto backend integra y estructura de manera exhaustiva la documentación desarrollada durante las Unidades 1 y 2.

DIAGRAMAS DEL SISTEMA

A continuación, se presenta el diagrama de arquitectura del módulo de autenticación y control de acceso de UTT Connect. El diagrama describe la interacción entre las capas del sistema durante el inicio de sesión mediante enlace mágico (magic link), desde la solicitud del usuario hasta la carga del dashboard correspondiente a su rol.

1. Arquitectura en cuatro capas

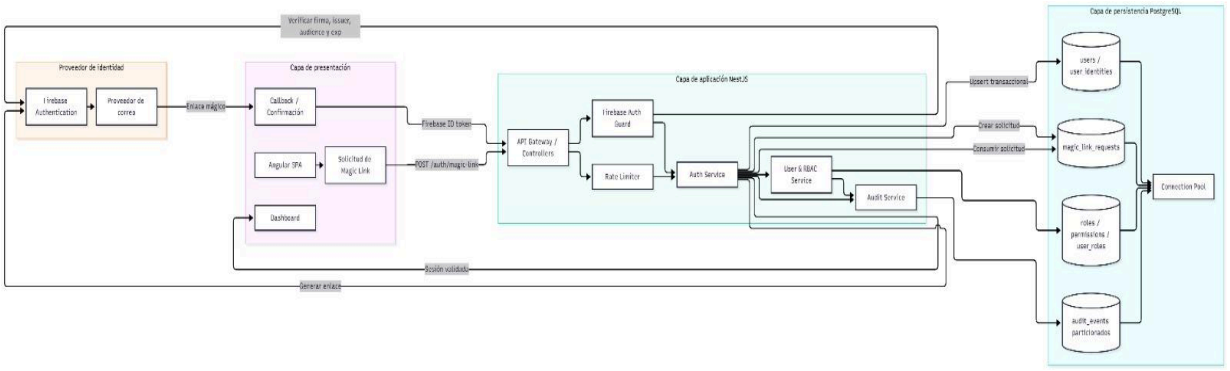


Figura 1. *Arquitectura en cuatro capas y flujo completo de autenticación con magic link.*

El diagrama muestra la separación de responsabilidades entre las siguientes capas:

- **Capa de presentación (Angular):** interfaz con la que interactúa la comunidad universitaria; captura el correo institucional, recibe el callback del enlace y muestra el dashboard de acuerdo con el rol del usuario.
- **Capa de aplicación (NestJS):** concentra la lógica de negocio del sistema mediante los servicios de autenticación, de usuarios y control de acceso (RBAC) y de auditoría.
- **Proveedor de identidad (Firebase):** servicio externo que emite el token de identidad del usuario, el cual es validado por la capa de aplicación.
- **Capa de persistencia (PostgreSQL):** almacena usuarios, identidades, enlaces de acceso, roles y permisos, así como los eventos de auditoría en la tabla particionada audit_events.

2. Flujo completo de autenticación

- **Paso 1. Solicitud de magic link:** el usuario ingresa su correo institucional en la aplicación Angular, que envía la solicitud al servicio de autenticación.
- **Paso 2. Envío de correo:** el sistema genera el enlace de acceso y lo envía al correo del usuario.
- **Paso 3. Callback con token Firebase:** al abrir el enlace, el usuario regresa a la aplicación con un token emitido por Firebase, que se envía al backend.
- **Paso 4. Validación:** el servicio de autenticación verifica la validez del token con el proveedor de identidad.
- **Paso 5. Upsert de usuario e identidad:** de manera transaccional, se crea o actualiza el registro del usuario y de su identidad en PostgreSQL.
- **Paso 6. Consumo del enlace:** el enlace queda registrado como consumido, evitando que pueda reutilizarse.
- **Paso 7. Resolución de roles y permisos:** el servicio de usuarios y RBAC determina el rol (Lector, Departamento o Administrador) y los permisos asociados.
- **Paso 8. Auditoría:** el servicio de auditoría registra el evento de inicio de sesión en audit_events.
- **Paso 9. Dashboard:** el sistema entrega la sesión al cliente, que presenta el dashboard correspondiente al rol del usuario.

3. Componentes clave

Componente	Responsabilidad
Auth Service	Orquesta la creación y el consumo del magic link, y realiza el upsert transaccional de usuario e identidad.
User & RBAC Service	Resuelve los roles y permisos del usuario autenticado para aplicar el control de acceso.

Audit Service	Registra los eventos de seguridad en la tabla audit_events, que se encuentra particionada.
Connection Pool	Punto único de acceso a PostgreSQL; todos los servicios consultan la base de datos a través de él.

4. Patrones de diseño aplicados

- **Arquitectura por capas y servicios:** cada servicio tiene una responsabilidad única, lo que facilita el mantenimiento y las pruebas de cada módulo.
- **Delegación de identidad:** la verificación de la identidad recae en Firebase, evitando almacenar y gestionar contraseñas propias.
- **Control de acceso basado en roles (RBAC):** los permisos se asignan a roles y no a usuarios individuales, lo que reduce el riesgo de fuga de permisos (R11).
- **Transacción atómica:** el upsert de usuario e identidad y el consumo del enlace se ejecutan como una unidad, garantizando consistencia de los datos.
- **Pool de conexiones:** centraliza y reutiliza las conexiones a PostgreSQL, controlando el consumo de recursos de la base de datos.
- **Bitácora de auditoría particionada:** audit_events conserva la trazabilidad de los accesos y mantiene el rendimiento de las consultas al crecer el volumen de registros.

5. Diagrama de casos de uso

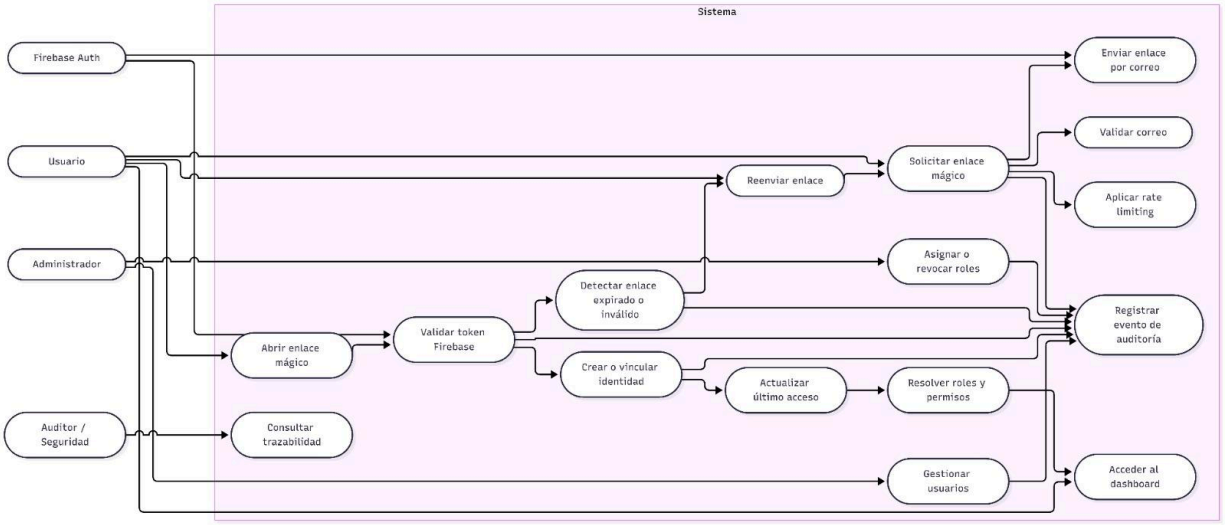


Figura 2. *Casos de uso del módulo de autenticación, identidad, administración y auditoría.*

El diagrama identifica cuatro actores que interactúan con el sistema:

- **Usuario:** miembro de la comunidad universitaria que solicita el acceso y utiliza el dashboard.
- **Administrador:** gestiona las cuentas de usuario y la asignación de roles.
- **Firebase Auth:** actor externo que emite y valida los tokens de identidad.
- **Auditor / Seguridad:** consulta la trazabilidad de los eventos del sistema.

Los casos de uso se agrupan en cuatro dominios funcionales:

Dominio	Casos de uso	Actores
Autenticación por magic link	Solicitar magic link; validar correo; aplicar rate limiting; enviar correo; abrir enlace; validar token Firebase; detectar enlace expirado o inválido; reenviar enlace.	Usuario, Firebase Auth
Identidad y acceso	Crear o vincular identidad; actualizar último acceso; resolver roles y permisos; acceder al dashboard.	Usuario
Administración	Gestionar usuarios; asignar y revocar roles.	Administrador
Auditoría	Consultar trazabilidad; registrar eventos (transversal a todos los flujos).	Auditor / Seguridad

6. Diagrama entidad-relación

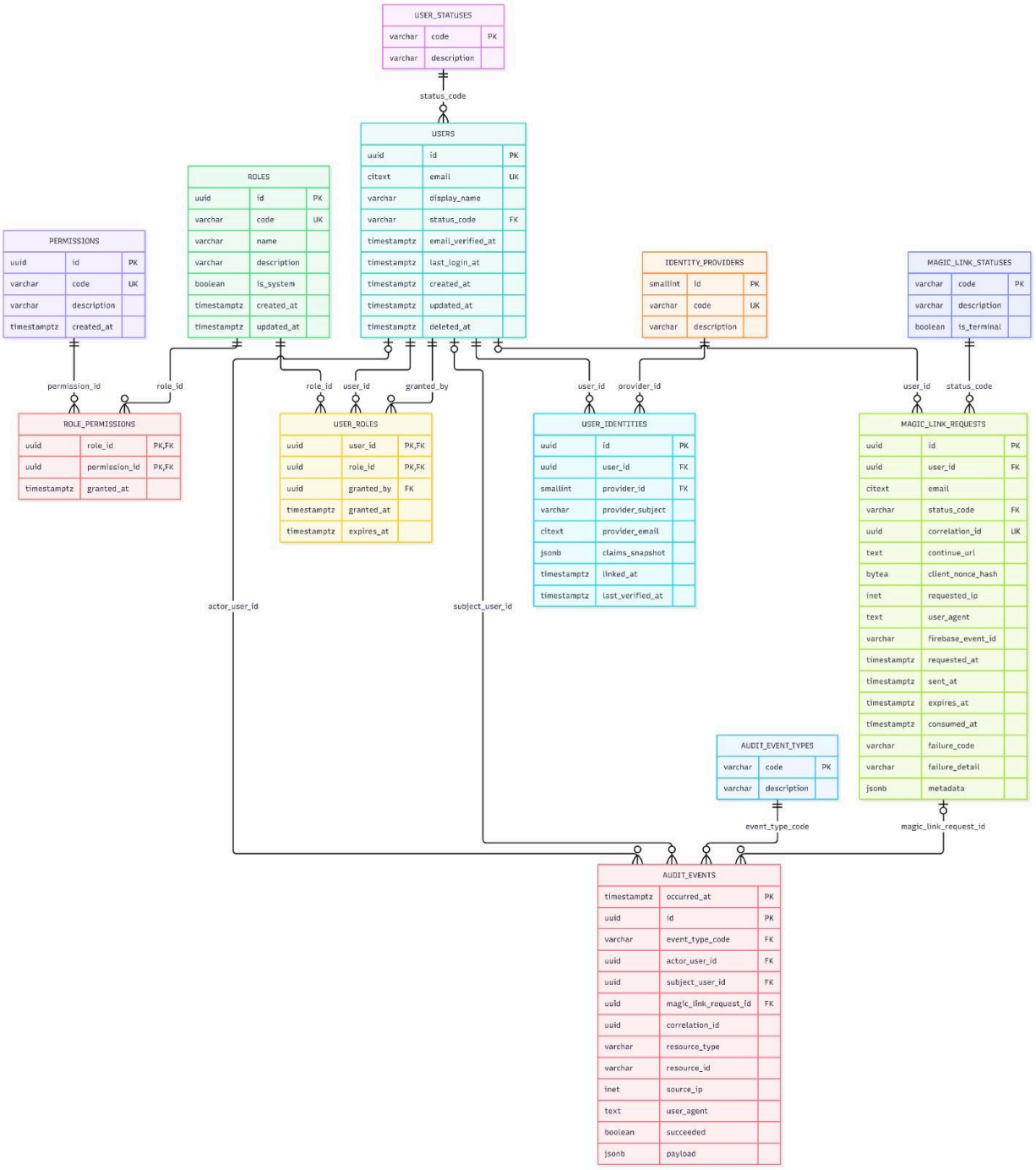


Figura 3. *Modelo entidad-relación de la base de datos en PostgreSQL.*

El modelo de datos se organiza en cinco grupos de entidades:

Grupo	Entidades	Propósito
Identidad	users, user_identities, identity_providers	Registro de usuarios, sus identidades y los proveedores que las respaldan.

Control de acceso (RBAC)	roles, permissions, role_permissions, user_roles	Definición de roles y permisos, y su asignación a los usuarios.
Magic link	magic_link_statuses, magic_link_requests	Solicitudes de enlace de acceso y su catálogo de estados.
Auditoría	audit_event_types, audit_events	Catálogo de tipos de evento y bitácora de eventos particionada.
Estados de usuario	user_statuses	Catálogo de estados posibles de una cuenta de usuario.

Las relaciones clave del modelo son las siguientes:

- **Usuario – identidades:** un usuario puede tener de una a muchas identidades (1..N).
- **Usuario – roles:** un usuario puede tener de uno a muchos roles (1..N) mediante user_roles.
- **Rol – permisos:** un rol agrupa de uno a muchos permisos (1..N) mediante role_permissions.
- **Usuario – magic links:** un usuario puede tener de cero a muchas solicitudes de magic link (0..N).
- **Auditoría:** cada evento de audit_events se vincula con el actor, el sujeto, el magic link involucrado y su tipo de evento.

7. Operación, respaldo y mantenimiento de la base de datos

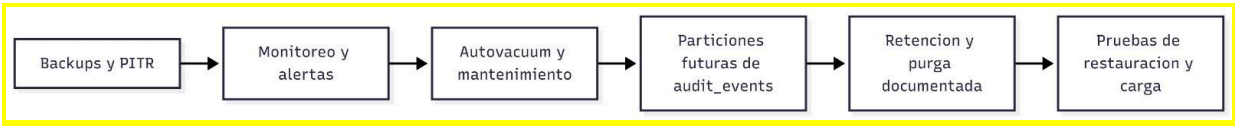


Figura 4. *Flujo de operación y mantenimiento de PostgreSQL.*

Para garantizar la disponibilidad, el rendimiento y la seguridad de PostgreSQL, se establece el siguiente flujo operativo:

- **Etapa 1. Respaldos y PITR:** copias de seguridad periódicas con recuperación a un punto en el tiempo (Point-in-Time Recovery).
- **Etapa 2. Monitoreo y alertas:** supervisión del estado de la base de datos con avisos ante fallas o consumo anormal de recursos.
- **Etapa 3. Autovacuum y mantenimiento:** limpieza automática de registros obsoletos y mantenimiento de las tablas e índices.
- **Etapa 4. Particiones futuras de audit_events:** creación anticipada de las particiones de la bitácora de auditoría.
- **Etapa 5. Retención y purga documentada:** política definida y documentada para conservar y eliminar los registros antiguos.
- **Etapa 6. Pruebas de restauración y carga:** verificación periódica de que los respaldos pueden restaurarse y de que el sistema soporta la carga esperada.

Adicionalmente, se aplican las siguientes medidas de seguridad de forma transversal:

- **Menor privilegio:** cada servicio accede a la base de datos únicamente con los permisos que necesita.

- **TLS y rotación de credenciales:** las conexiones se cifran en tránsito y las credenciales se renuevan periódicamente.
- **Enmascarado de PII en logs:** los datos personales identificables se ocultan en los registros del sistema.

DICCIONARIO DE DATOS

A continuación, se detalla la estructura principal de la base de datos para garantizar la integridad referencial y cumplir con el diseño relacional propuesto.